VIETTEL DIGITAL TALENT 2026

Streaming Processing in BigData Platform

Real-time data pipelines with Apache Kafka & Apache Flink

Mentor: Nguyen Minh Hieu
Student: Ngo Quang Hieu

Data Engineer Track
Viettel Digital Talent 2026

July 1^{st}, 2026

Table of Contents

viettel
Theo cách của bạn

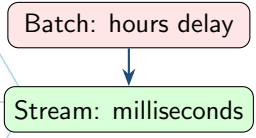
Table of Contents

viettel
Theo cách của bạn

Why Stream Processing?

The Problem

- **High latency** — jobs run hourly/daily, so insights lag reality by hours
- **Stale decisions** — by the time data is processed, the moment to act has passed
- **Resource spikes** — large periodic loads strain infra, then sit idle



Stream Processing Enables

- **Live monitoring & alerting** — detect anomalies or failures as they happen, not next morning
- **Continuous aggregation** — dashboards show current counts/metrics, not a stale snapshot
- **Smooth, steady load** — events processed as they arrive, avoiding the resource spikes of periodic batch runs

Objectives

Objectives

1. Overview **Apache Kafka** as message queue
2. Compare **Spark Streaming vs Flink** for streaming processing
3. Build ELT streaming pipeline
4. Demonstrate real-time dashboard with live metrics

Table of Contents

Why Apache Kafka?

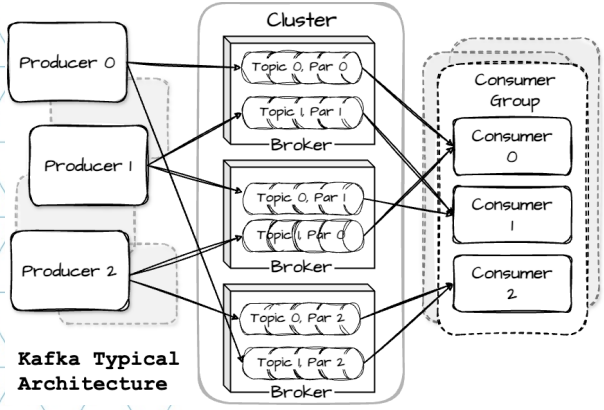
Problems Without a Message Queue

- **Tight coupling** — producer knows every consumer
- **No buffering** — slow consumer $\rightarrow$ data loss
- **No replay** — consumed $=$ gone forever
- **Fan-out cost** — N consumers $=$ N calls

How Kafka Solves It

- **Decoupling** — write to topic, subscribe independently
- **Durable buffer** — persisted to disk, replicated
- **Replay** — configurable retention; re-read anytime
- **Scalability** — partitions scale throughput linearly
- **Fan-out free** — multiple groups, own offsets

Kafka Architecture



Kafka Typical Architecture

- **Producer** — partitions by key; batches & compresses; `acks=all` for durability
- **Broker** — leader/follower replicas; append-only log; sequential I/O
- **Topic** — immutable records; time-based or log-compaction retention
- **Partition** — ordered, offset-indexed; unit of parallelism
- **Consumer Group** — one partition → one member; independent offset per group

Max parallelism = number of partitions

Table of Contents

viettel
Theo cách của bạn

Batch vs. Stream Processing

Batch Processing

- Collect → store → process in bulk
- Latency: **hours to days**
- Tools: Hadoop, Spark, scheduled SQL

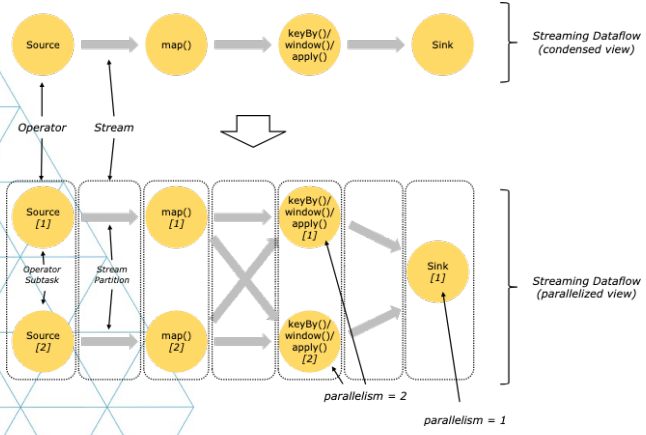
Stream Processing

- Compute *continuously* as records arrive
- Latency: **milliseconds to seconds**
- Tools: Apache Flink, Spark Streaming

Key Challenges in Streaming

1. **Unbounded data** — stream never ends; state must be bounded
2. **Out-of-order events** — need watermarks to handle late arrivals
3. **Latency** — must process quickly to keep up with incoming data
4. **Fault tolerance** — recover without loss or duplicates; exactly-once semantics
5. **State & Checkpoint Management** — large state must be checkpointed to durable storage (e.g., local, cloud)

DAG-based Dataflow: Shared Foundation

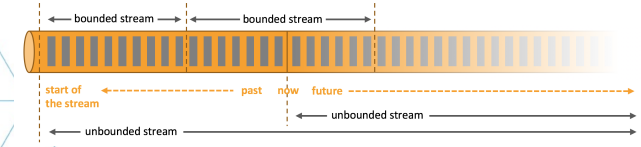


Both Spark and Flink represent computation as a Directed Acyclic Graph (DAG)

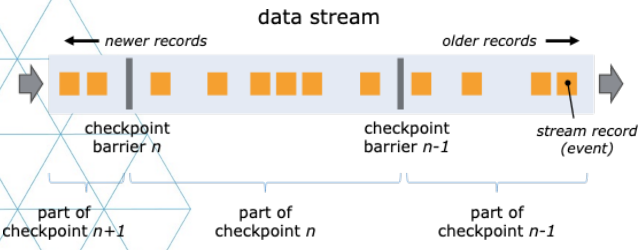
What is a Dataflow DAG?

- **Directed** — data flows one way; no cycles
- **Acyclic** — guarantees deterministic, terminating execution
- **Graph** — **nodes** are operators (map, filter, join, aggregate); **edges** are data streams between them
- Each operator processes records and passes results downstream
- Operators at the same level run **in parallel** across partitions
- Both Spark & Flink compile your code into a DAG — they differ in *when* and *how* they execute it

Apache Flink: True Event Streaming



True streaming: records flow through operator DAG one-by-one



Chandy-Lamport barriers: async snapshots without pausing processing

Dataflow Model

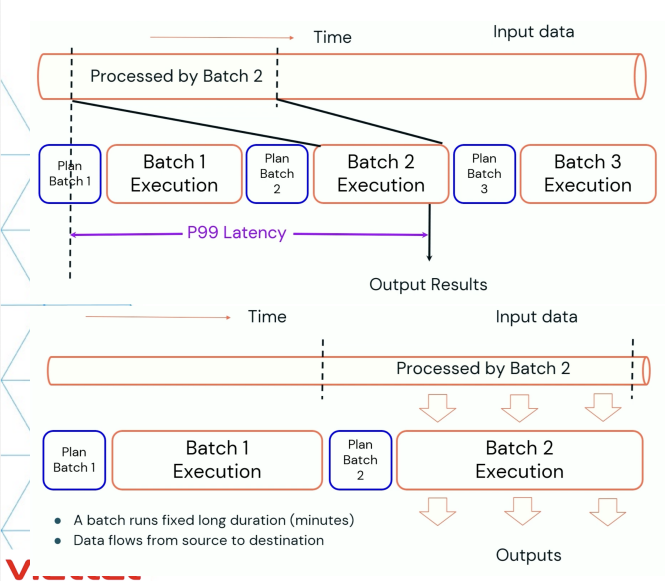
- **Record-at-a-time** — no batching delay
- Operators form a DAG; data flows continuously
- **Event time + watermarks** — correct out-of-order handling

Fault Tolerance

- Barrier checkpoints injected into streams like a conveyor belt
- **Exactly-once** with transactional sinks
- State management can be store in heap or RockDB
- Checkpoint is fully async, no pause in processing (local or cloud)

viettel
Theo cách của bạn

Apache Spark Structured Streaming



Micro-batch (default)

- Latency: **100 ms – seconds**
- Still depend on exist batch mode stage, plan create a lot of latency
- State management In-memory+WAL or RocksDB
- Checkpoint sync write to checkpoint location (local or cloud) create latency

Real-time Mode (Spark 4)

- Inherit from micro-batch, but update **concurrent stages** and **longer duration micro-batches**
- Mitigate the latency of micro-batch in planning phase and sequential stages
- Latency: **sub-100 ms**

Spark vs. Flink: Side-by-Side

Dimension	Apache Spark	Apache Flink
Execution model	Micro-batch (scheduled triggers); Real-time Mode (Spark 4)	True record-at-a-time streaming
Latency	100 ms–s (micro-batch); sub-100 ms (RTM)	1–10 ms
State management	In-memory / RocksDB (Spark 3.2+); managed per trigger	RocksDB or heap; always-live, keyed state
Checkpoint	WAL per micro-batch	Async Chandy-Lamport barriers; incremental snapshots
Use case	Unified batch + streaming	Low-latency, stateful, CEP, event-time joins

Table of Contents

Why Streamify?

The Use Case

- Music streaming platforms generate **continuous, high-velocity events** — every play, skip, pause, and login is an event
- Users expect **live charts** and real-time fraud detection — batch pipelines cannot deliver this
- A realistic domain to validate all theoretical concepts end-to-end

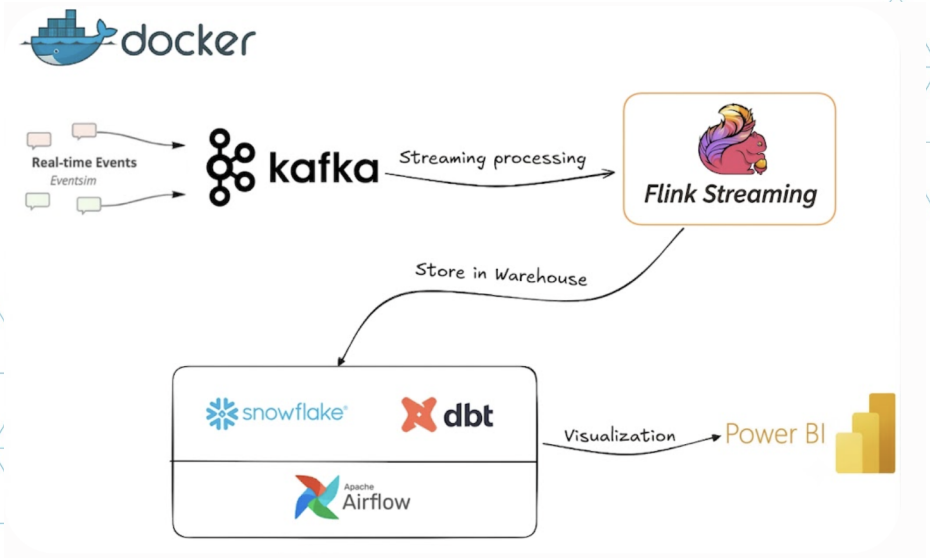
3 Event Types Simulated

- `listen_events` — song plays, skips, durations
- `page_view_events` — user navigation behaviour
- `auth_events` — logins, session starts/ends

What Streamify Validates

- **Kafka** — durable, decoupled event bus between producer and all downstream consumers
- **PyFlink** — low-latency, exactly-once ingestion into Snowflake
- **dbt** — transforms raw staging data into an analytics-ready star schema
- **Airflow** — orchestrates transformation runs reliably
- **Power BI** — surfaces live insights from a pre-joined wide view

Streamify: End-to-End Architecture



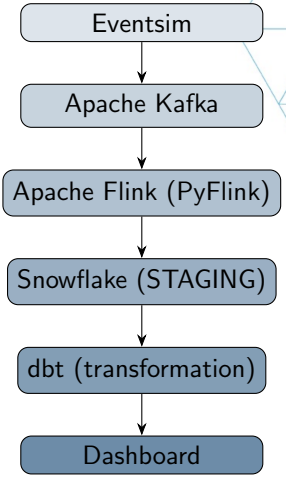
Pipeline: Ingest → Process → Model → Serve

Real-time Ingestion

- **Eventsim** — generates 3 Kafka topics: `listen_events`, `page_view_events`, `auth_events`
- **Kafka** — durable, partitioned transport
- **PyFlink** — parses JSON, enriches timestamps,
  writes to Snowflake via JDBC (exactly-once)

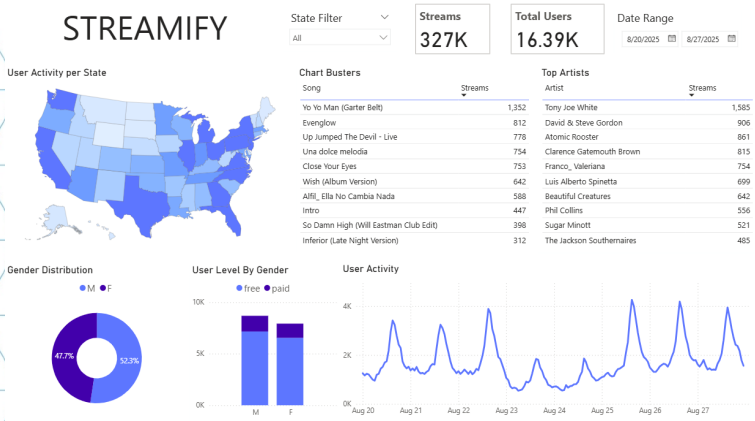
Transformation & Orchestration

- **dbt** — builds star schema inside Snowflake (fact_streams + 5 dim tables + wide_streams view)
- **Airflow** — schedules dbt run via DockerOperator



viettel
Theo cách của bạn

Dashboard: Real-time Insights



Key Metrics

- Top artists by play count
- Songs played over time (hourly)
- Geographic distribution (US map)
- Free vs. paid listener split

viettel
Theo cách của bạn

Table of Contents

viettel
Theo cách của bạn

Conclusion

Key Takeaways

1. **Kafka** decouples producers from consumers — swap the processing engine without touching the rest of the pipeline
2. **Flink** delivers sub-10 ms latency, native event-time semantics, and exactly-once guarantees — outperforming Spark for stateful real-time workloads
3. **ELT pattern** keeps concerns separate: Flink loads raw events, dbt transforms inside Snowflake, Airflow orchestrates

Future Directions

- **Cloud** — migrate to Kinesis + Confluent Cloud
- **Schema evolution** — Confluent Schema Registry + Avro

- THE END -

Thank you for your attention

Contact:
Ngo Quang Hieu — Viettel Digital Talent 2026